Demystifying

Spring Boot Magic

Patrick Baumgartner
42talents GmbH, Zürich, Switzerland

@patbaumgartner
patrick.baumgartner@42talents.com

Abstract

Demystifying Spring Boot Magic

One of Spring Boot's most powerful features is its auto-configuration. This magic is key to the convention-over-configuration approach that has brought a massive boost in productivity to Java developers. But is it magic?

In this talk, we will walk through how auto-configuration works in detail. Features like AutoConfiguration.imports, @AutoConfiguration, @Conditional, @AutoConfigureBefore and many more.

We will take a look at how you could write a Spring Boot starter yourself and elaborate on why this might make sense for you in your current project/company.

DEMYSTIFYING Spring Boot Magic

Patrick Baumgartner
@patbaumgartner

Introduction

Patrick Baumgartner

Technical Agile Coach @ **42talents**

My focus is on the **development of software solutions *with* humans**.

Working with Spring since 2008 and teaching courses since 2011.

Lecturer @ **Zurich University of Applied Sciences ZHAW**

Co-Organizer of Voxxed Days Zurich, **JUG Switzerland**, **Java Champion**, Oracle ACE Pro Java ...

@patbaumgartner

I
DO
NOT
CARE

!Agenda

- Inversion of Control (IoC)
- Spring Beans
- Configuration Styles
  - XML
  - ComponentScan
  - Java Config
- Spring Bean Lifecycle
- Spring AOP
- Composed Annotations

Agenda

Agenda

- Magic vs Boilerplate
- Dependency Version Management
- Spring Boot Starters
- AutoConfiguration
- Property Sources
- FailureAnalyzer
- `@SpringBootApplication & @SpringBootTest`
- Fat JARs & Build Packs
- Custom Starters

Magic vs Boilerplate

What Developers Mean by "Magic"

- Code behavior at runtime is unclear from reading the source alone.
- Violations of *Single Responsibility* or *Command/Query Separation* often feel like "magic".
- The line between "magic" and understandable code is thin. It’s often about how surprising the behavior is.

What Developers Mean by "Boilerplate"

- Repetitive code written again and again across applications.
- Often duplicated within the same codebase.
- Routine and uninteresting, yet often necessary.

Magic or Boilerplate?

Magic or Boilerplate?

- Both "magic" and "boilerplate" are subjective terms.
- Neither is inherently bad, they depend on context and trade-offs.
- *Boilerplate is not the opposite of magic!*

Too Much Magic

“Boilerplate is mostly a problem when writing code. Magic when reading & debugging. We read & debug more than we write.”

David Tanzer - @dtanzer

When Magic Becomes Knowledge

"It is too much magic until you read the docs, several books, and the source code, after that, the magic becomes knowledge."

Unknown

Spring Boot Philosophy

- An opinionated approach to building **production-ready** Spring applications.
- Favors **convention over configuration** to get you running quickly.
- Helps developers **boot up Spring applications with minimal setup**.

SNAKE
OIL

Spring Boot Dependency Management

Spring Boot Parent POM

- Defines versions of key dependencies.
- Uses `dependencyManagement` internally via the `spring-boot-dependencies` parent.
- Preconfigures common Maven plugins.
- Sets the Java version for the project.

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent<artifactId>
  <version>3.5.0</version>
</parent>
```

Spring Boot “Starter” Dependencies

- Provide an easy way to include multiple coordinated dependencies.
- Automatically pull in required **transitive dependencies**.

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <!-- NO VERSION NEEDED -->
  </dependency>
</dependencies>
```

- Versions defined in `spring-boot-dependencies`

```
<tomcat.version>10.1.40</tomcat.version>
```

See also: https://repo1.maven.org/maven2/org/springframework/boot/spring-boot-dependencies/3.5.0/spring-boot-dependencies-3.5.0.pom

Dependency Management with Gradle

- The Dependency Management Plugin ensures consistent, predefined versions for project dependencies.

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '3.5.0'
    id 'io.spring.dependency-management' version '1.1.7'
}
```

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
```

```
ext['tomcat.version'] = '10.1.40'
```

@SpringBootApplication

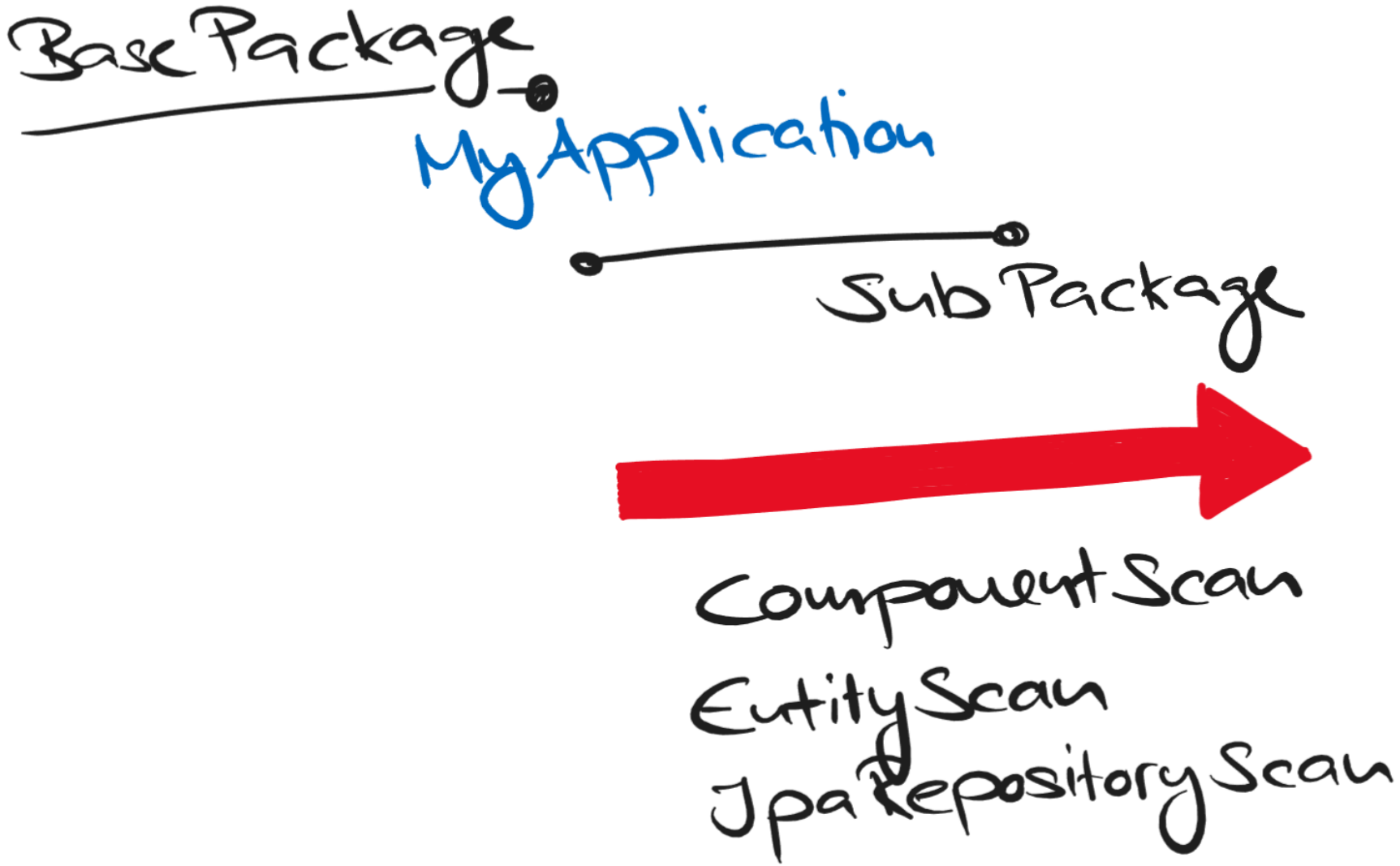
Auto Configuration

- Follows the convention over configuration principle.
- Can feel like "magic" due to hidden behaviors.

```
@SpringBootApplication
public class MyApplication { }
```

- Is shorthand for:

```
@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan
public class MyApplication { }
```

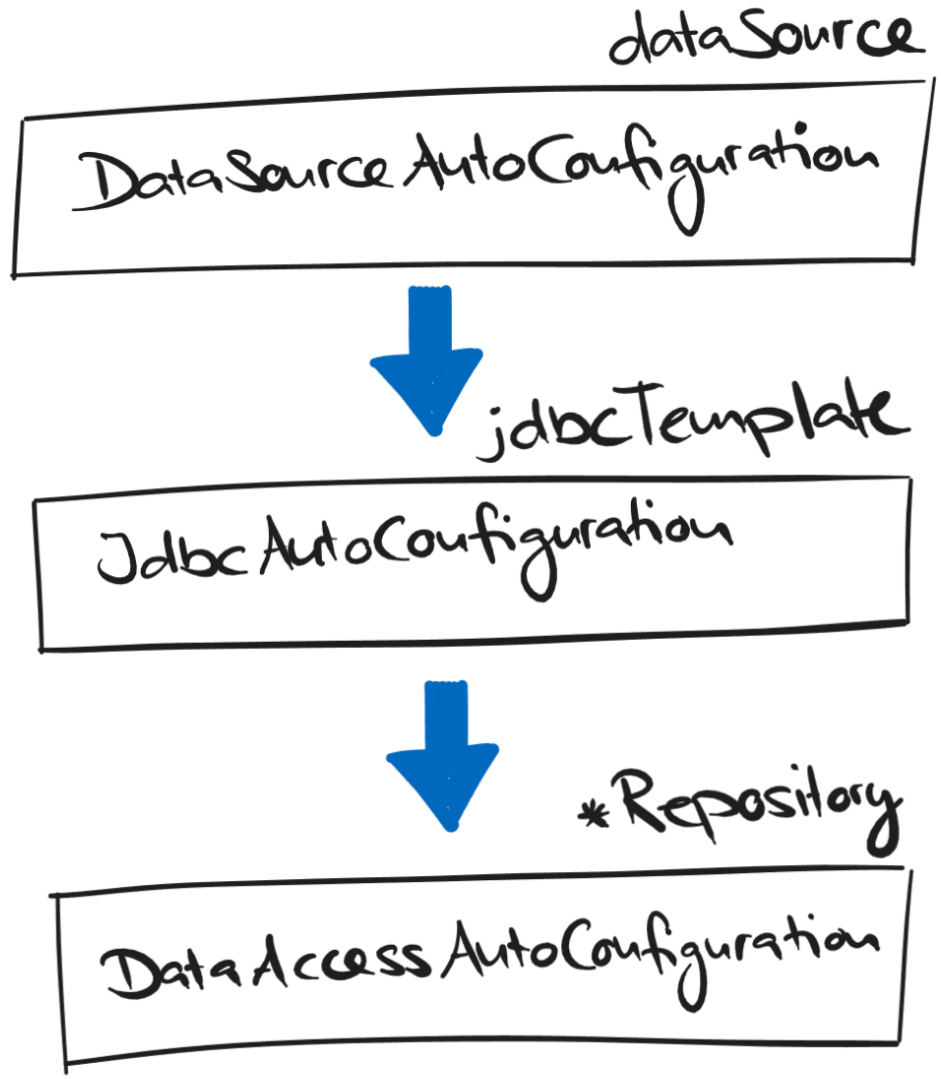

Base Package
My Application
Sub Package
Component Scan
Entity Scan
Jpa Repository Scan

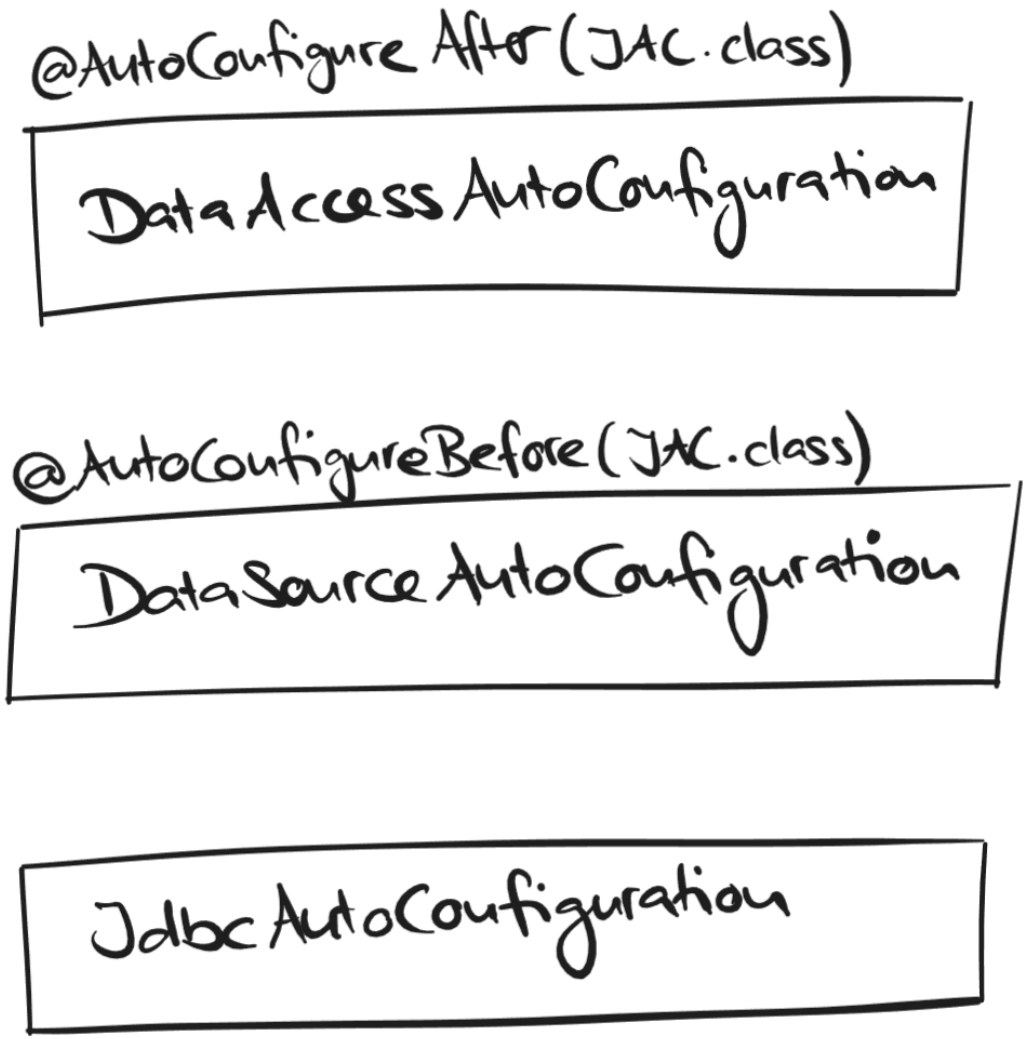
@AutoConfiguration or "@Abrakadabra"

@AutoConfiguration

```
...
@Configuration(proxyBeanMethods = false)
@AutoConfigureBefore
@AutoConfigureAfter
public @interface AutoConfiguration {

        @AliasFor(annotation = Configuration.class)
        String value() default "";

        @AliasFor(annotation = AutoConfigureBefore.class, attribute = "value")
        Class<?>[] before() default {};

        @AliasFor(annotation = AutoConfigureBefore.class, attribute = "name")
        String[] beforeName() default {};

        @AliasFor(annotation = AutoConfigureAfter.class, attribute = "value")
        Class<?>[] after() default {};

        @AliasFor(annotation = AutoConfigureAfter.class, attribute = "name")
        String[] afterName() default {};
}
```

@AutoConfigureBefore / @AutoConfigureAfter


dataSource
DataSourceAutoConfiguration
jdbcTemplate
JdbcAutoConfiguration
*Repository
DataAccessAutoConfiguration


@AutoConfigureAfter(JAC.class)
DataAccessAutoConfiguration
@AutoConfigureBefore(JAC.class)
DataSourceAutoConfiguration
JdbcAutoConfiguration

Import Candidates in META-INF/spring/

org.springframework.boot.autoconfigure.AutoConfiguration.imports

```
...
org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
org.springframework.boot.autoconfigure.jdbc.JdbcClientAutoConfiguration
org.springframework.boot.autoconfigure.jdbc.JdbcTemplateAutoConfiguration
org.springframework.boot.autoconfigure.jdbc.JndiDataSourceAutoConfiguration
org.springframework.boot.autoconfigure.jdbc.XADataSourceAutoConfiguration
org.springframework.boot.autoconfigure.jdbc.DataSourceTransactionManagerAutoConfiguration
...
org.springframework.boot.autoconfigure.jooq.JooqAutoConfiguration
...
org.springframework.boot.autoconfigure.liquibase.LiquibaseAutoConfiguration
org.springframework.boot.autoconfigure.mail.MailSenderAutoConfiguration
org.springframework.boot.autoconfigure.mail.MailSenderValidatorAutoConfiguration
...
org.springframework.boot.autoconfigure.orm.jpa.HibernateJpaAutoConfiguration
org.springframework.boot.autoconfigure.pulsar.PulsarAutoConfiguration
org.springframework.boot.autoconfigure.pulsar.PulsarReactiveAutoConfiguration
org.springframework.boot.autoconfigure.quartz.QuartzAutoConfiguration
org.springframework.boot.autoconfigure.r2dbc.R2dbcAutoConfiguration
org.springframework.boot.autoconfigure.r2dbc.R2dbcProxyAutoConfiguration
org.springframework.boot.autoconfigure.r2dbc.R2dbcTransactionManagerAutoConfiguration
```

Auto Configuration

```
@AutoConfiguration(before = SqlInitializationAutoConfiguration.class)
@ConditionalOnClass({ DataSource.class, EmbeddedDatabaseType.class })
@ConditionalOnMissingBean(type = "io.r2dbc.spi.ConnectionFactory")
@EnableConfigurationProperties(DataSourceProperties.class)
@Import({ DataSourcePoolMetadataProvidersConfiguration.class, DataSourceCheckpointRestoreConfiguration.class })
public class DataSourceAutoConfiguration {

        @Configuration(proxyBeanMethods = false)
        @Conditional(EmbeddedDatabaseCondition.class)
        @ConditionalOnMissingBean({ DataSource.class, XADataSource.class })
        @Import(EmbeddedDataSourceConfiguration.class)
        protected static class EmbeddedDatabaseConfiguration {
        }

        @Configuration(proxyBeanMethods = false)
        @Conditional(PooledDataSourceCondition.class)
        @ConditionalOnMissingBean({ DataSource.class, XADataSource.class })
        @Import({ DataSourceConfiguration.Hikari.class, DataSourceConfiguration.Tomcat.class,
                        DataSourceConfiguration.Dbcp2.class, DataSourceConfiguration.OracleUcp.class,
                        DataSourceConfiguration.Generic.class, DataSourceJmxConfiguration.class })
        protected static class PooledDataSourceConfiguration {
```

Conditions

- Configuration with `@Conditional`
  - OnClass / OnMissingClass
  - OnBean / OnMissingBean
  - OnProperty
  - OnResource
  - OnExpression
  - OnJava
  - OnSingleCandidate
  - OnWarDeployment / OnNotWarDeployment
  - OnWebApplication / OnNotWebApplication
  - OnCloudPlatform
  - ...

@Profile

- Controls bean registration based on the active environment profile.
- Internally uses `ProfileCondition` to check for matching profiles.

```
@Target({ElementType.TYPE, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Conditional(ProfileCondition.class)
public @interface Profile {

        String[] value();

}
```

Externalized Configuration

Externalized Configuration (Simplified)

- Configuration properties loaded in a specific order. Highest priority wins:

1. Devtools settings
2. `@TestPropertySource` and `@SpringBootTest` properties
3. Command line arguments
4. `SPRING_APPLICATION_JSON` (inline JSON properties).
5. ServletConfig / ServletContext parameters
6. JNDI attributes from `java:comp/env`
7. Java System properties (`System.getProperties()`)
8. OS environment variables
9. Profile-specific application properties
10. Application properties / YAML
11. `@PropertySource` files
12. `SpringApplication.setDefaultProperties`

@ConfigurationProperties

@ConfigurationProperties

- Binds external properties to Spring beans using a *prefix*, reducing the need for repetitive `@Value("${…}")` usage.

```
@ConfigurationProperties(prefix = "product")
public class RoutingDataSourceTenantProperties {

  private List<String> tenants = new ArrayList<>();

  public List<String> getTenants() {
    return tenants;
  }
}
```

```
product.tenants[0]=tenant1
product.tenants[1]=tenant2
product.tenants[2]=tenant3
```

Relaxed Binding in @ConfigurationProperties

- Supports flexible mapping between property keys and Java fields.

```
@ConfigurationProperties("tenant.client-connection")
public class ConnectionSettings {
    private String hostUrl;
    …
}
```

- Valid property keys for `hostUrl` :
  - `tenant.clientConnection.hostUrl`
  - `tenant.client-connection.host-url`
  - `tenant.client_connection.host_url`
  - `TENANT_CLIENTCONNECTION_HOSTURL`

@SpringBootTest

@SpringBootTest

- Enables integration tests with full Spring Boot context.
- Adds features beyond the Spring TestContext Framework:
  - Uses `SpringBootContextLoader` by default unless overridden.
  - Auto-detects a `@SpringBootConfiguration` if no nested `@Configuration` or explicit classes are provided.
  - Supports custom `Environment` properties.
  - Allows defining application arguments.
  - Supports different `webEnvironment` modes, including launching an embedded web server on a defined or random port.

@SpringBootTest

```
@BootstrapWith(SpringBootTestContextBootstrapper.class)
@ExtendWith(SpringExtension.class)
public @interface SpringBootTest {

        @AliasFor("properties")
        String[] value() default {};

        @AliasFor("value")
        String[] properties() default {};

        String[] args() default {};
        Class<?>[] classes() default {};
        WebEnvironment webEnvironment() default WebEnvironment.MOCK;
        UseMainMethod useMainMethod() default UseMainMethod.NEVER;
}
```

Note: Use @SpringBootTest judiciously, overuse can significantly slow down your test suite.

FailureAnalyzer

Exceptions all the way ...

```
at org.springframework.beans.factory.support.DefaultListableBeanFactory.doResolveDepe...
at org.springframework.beans.factory.support.DefaultListableBeanFactory.resolveDepend...
at org.springframework.beans.factory.annotation.AutowiredAnnotationBeanPostProcessor$...
... 226 common frames omitted
Caused by: org.springframework.beans.factory.BeanCreationException: Could not autowire fi...
at org.springframework.beans.factory.annotation.AutowiredAnnotationBeanPostProcessor$...
at org.springframework.beans.factory.annotation.InjectionMetadata.inject(InjectionMet...
at org.springframework.beans.factory.annotation.AutowiredAnnotationBeanPostProcessor....
... 246 common frames omitted
Caused by: org.springframework.beans.factory.BeanCurrentlyInCreationException: Error crea...
at org.springframework.beans.factory.support.DefaultSingletonBeanRegistry.beforeSingl...
at org.springframework.beans.factory.support.DefaultSingletonBeanRegistry.getSingleto...
at org.springframework.beans.factory.support.AbstractBeanFactory.doGetBean(AbstractBe...
at org.springframework.beans.factory.support.AbstractBeanFactory.getBean(AbstractBean...
at org.springframework.beans.factory.support.DefaultListableBeanFactory.findAutowireC...
at org.springframework.beans.factory.support.DefaultListableBeanFactory.doResolveDepe...
at org.springframework.beans.factory.support.DefaultListableBeanFactory.resolveDepend...
at org.springframework.beans.factory.annotation.AutowiredAnnotationBeanPostProcessor$...
... 248 common frames omitted
```

FailureAnalyzer

- Handles specific exceptions during Spring Boot application startup, such as: `NoSuchBeanDefinition`, `Bind`, `BeanCurrentlyInCreation`, `PortInUse`, `DataSourceBeanCreation`, etc.
- Write your own `CustomFailureAnalyzer`.

```
Error starting ApplicationContext. To display the condition evaluation report re-run your application with 'debug' enabled.
2025-05-18T18:44:28.721+02:00 ERROR 43696 --- [my-application] [           main] o.s.b.d.LoggingFailureAnalysisReporter   :

***************************
APPLICATION FAILED TO START
***************************

Description:

Web server failed to start. Port 8080 was already in use.

Action:

Identify and stop the process that's listening on port 8080 or configure this application to listen on another port.
```

Fat JARs

Fat JARs

- The concept isn't new:
  - Also known as Uber-JAR or One-JAR.
  - Popularized by tools like the Apache Maven Shade Plugin.
- Spring Boot simplifies it via the Spring Boot Maven Plugin:

```
<plugin>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>
    <version>3.5.0</version>
    <executions>
        <execution>
            <goals>
                <goal>repackage</goal>
            </goals>
        </execution>
    </executions>
</plugin>
```

Exploded Fat JARs

- Fat JARs can be an anti-pattern in containerized environments.
- Lead to fat image layers instead of thin, cacheable layers.
- Thin layers reduce build time, network traffic, and improve Docker cache efficiency.

```
$ du -sh target/demo-0.0.1-SNAPSHOT*
14M  target/demo-0.0.1-SNAPSHOT.jar
4.0K target/demo-0.0.1-SNAPSHOT.jar.original
```

- Layertools allow you to extract the layers from a fat JAR.

```
java -Djarmode=tools -jar demo-0.0.1-SNAPSHOT.jar extract --layers --launcher
```

paketo

buildpacks

Buildpacks FTW!

- The Spring Boot Maven Plugin uses Cloud Native Buildpacks via the `build-image` goal.
- Automatically detects the Spring Boot app and optimizes the container image.
- **Runtime optimizations:**
  - Explodes the fat JAR for better layer granularity.
  - Applies resource tuning (e.g. memory settings) at startup.
- **Image optimizations:**
  - Adds base layers from Buildpacks and Spring Boot.
  - Supports AOT and CDS for faster startup.
  - **Incremental builds:** Only changed layers are rebuilt.

Spring Boot DevTools

Spring Boot DevTools

- Applies development-friendly defaults (e.g. disables caching).
- Automatically restarts the application on file changes.
- Starts an embedded LiveReload server.
- Supports a global config in `$HOME/.config/spring-boot`:
  - `spring-boot-devtools.properties`, `.yaml`, or `.yml`
- Enables remote update and debugging over HTTP.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <optional>true</optional>
</dependency>
```

Community

JOIN THE CONVERSATION

Spring Framework

on Bluesky

Recap

Recap

AutoConfiguration is not magic!

- `@SpringBootApplication` enables auto-configuration.
- Uses ImportCandidates instead of classpath scanning.
- `@ConditionalOnClass` and `@ConditionalOnBean` enable flexible, overrideable behavior.
- Auto-configurations run after user-defined beans.
- Control ordering with `@AutoConfigureBefore` and `@AutoConfigureAfter`.
- Inspect active auto-configurations using the `--debug` flag.

Creating a Custom Starter

Why Create a Custom Starter?

You might want to bundle things like a **custom logging setup**, **consistent security rules**, **integration test scaffolding**, or **company-wide metrics and monitoring** into a reusable Spring Boot component.

Creating a custom starter helps you:

- **Standardize configuration** across projects
- **Package reusable functionality** into one dependency
- **Simplify dependency and version management**
- **Enable plug-and-play features** via auto-configuration
- **Improve developer experience** with minimal setup

Step-by-Step: Creating a Custom Starter

We will build a custom Spring Boot starter in 5 steps:

1. Define modules
2. `@ConfigurationProperties` and `spring-boot-configuration-processor`
3. `@AutoConfiguration` and `@Conditional`
4. Register auto-configuration
5. Package as a starter

Step 1: Define Modules

Split the starter into two Maven modules:

- `{module}-spring-boot-autoconfigure` : Contains the `@AutoConfiguration` and properties classes.
- `{module}-spring-boot-starter{-optional postfix}` : Brings together dependencies from the autoconfigure module and the core library.

The Autoconfigure Module

The Autoconfigure Module

- Custom module should be named like
  - `{module}-spring-boot-autoconfigure`
- Contains
  - Java Classes
    - `{Module}Properties` which will enable setting custom properties through `application.properties` or `application.yml`
    - `{Module}AutoConfiguration` which will create the spring beans
  - `org.springframework.boot.autoconfigure.AutoConfiguration.impor` in `META-INF/spring/`
    - Pointing to the `{Module}AutoConfiguration`
  - `spring-boot-configuration-processor`
    - Generating IDE support for auto completion

Step 2: @ConfigurationProperties

- Used to make properties from `application.properties` or `application.yml` accessible in the code

```java
@Data
@ConfigurationProperties(prefix = "javafaker")
public class JavaFakerProperties {

    /**
    * Locale in the format of de-CH, higher precendence over language.
    */
    private String locale;


    /**
    * Language in the 2 character format like de.
    */
    private String language;
}
```

```
#application.properties from the business application
javafaker.language=de
javafaker.locale=de-CH
```

Step 2: spring-boot-configuration-processor

- Generates `META-INF/spring-configuration-metadata.json` for the IDEs.

```
{
  "hints": [],
  "groups": [
    {
      "sourceType": "com.fortytwotalents.javafaker.autoconfigure.JavaFakerProperties",
      "name": "javafaker",
      "type": "com.fortytwotalents.javafaker.autoconfigure.JavaFakerProperties"
    }
  ],
  "properties": [
    {
      "sourceType": "com.fortytwotalents.javafaker.autoconfigure.JavaFakerProperties",
      "name": "javafaker.language",
      "description": "language in the 2 character format like de.",
      "type": "java.lang.String"
    },
    {
      "sourceType": "com.fortytwotalents.javafaker.autoconfigure.JavaFakerProperties",
      "name": "javafaker.locale",
```

Step 3: AutoConfiguration & @Conditional

```
@AutoConfiguration
@ConditionalOnClass(Faker.class)
@EnableConfigurationProperties(JavaFakerProperties.class)
public class JavaFakerAutoConfiguration {

    @Autowired
    private JavaFakerProperties fakerProperties;

    @Bean
    @ConditionalOnMissingBean
    public Faker faker() {
        if (!StringUtils.isEmpty(fakerProperties.getLocale())) {
            return Faker.instance(new Locale(fakerProperties.getLocale()));
        }

        if (!StringUtils.isEmpty(fakerProperties.getLanguage())) {
            return Faker.instance(new Locale(fakerProperties.getLanguage()));
        }

        return Faker.instance();
    }
}
```

Step 4: Register Auto-Configuration

- While startup, Spring Boot looks for all files named `org.springframework.boot.autoconfigure.AutoConfiguration.imports` in the classpath to trigger auto-configuration
- File is located in the `META-INF/spring` directory
  - e.g. from the `javafaker-spring-boot-autoconfigure` project

```
# org.springframework.boot.autoconfigure.AutoConfiguration.imports

# Import Candidates
com.fortytwotalents.javafaker.autoconfigure.JavaFakerAutoConfiguration
```

The Starter Module

Step 5: Package the Starter

In the starter module:

- Add a dependency on the autoconfigure module
- Include any required external libraries
- Ensure the `starter` module has no Spring logic itself

```
<dependency>
  <groupId>com.fortytwotalents</groupId>
  <artifactId>javafaker-spring-boot-autoconfigure</artifactId>
  <version>${project.version}</version>
</dependency>
<dependency>
  <groupId>com.github.javafaker</groupId>
  <artifactId>javafaker</artifactId>
  <version>${java-faker.version}</version>
</dependency>
```

My Application

Using the Starter

Now, consumers only need to add the starter dependency.

```xml
<dependency>
  <groupId>com.fortytwotalents</groupId>
  <artifactId>javafaker-spring-boot-starter</artifactId>
  <version>0.0.1-SNAPSHOT</version>
</dependency>
```

```java
@Bean
public CommandLineRunner commandLineRunner(Faker faker) {
  return args -> {
    log.info("Faker: {}", faker);
    log.info("Name: {}", faker.name().fullName());
    log.info("First Name: {}", faker.name().firstName());
    log.info("Last Name: {}", faker.name().lastName());
    log.info("Street Address: {}", faker.address().streetAddress());
  };
}
```

```properties
#application.properties from the business application
javafaker.language=de
javafaker.locale=de-CH
```

THANKS!!!

Patrick Baumgartner
@patbaumgartner

DEMYSTIFYING SPRING BOOT'S AUTO-CONFIGURATION MAGIC [WORKSHOP]

TODAY
16:30 - 18:20

Fabian Krüger
@hiiamfabian

Moritz Halbritter
@m_halbritter